

Tugas Besar 2

IF3070 Dasar Inteligensi Artifisial

Implementasi Algoritma Pembelajaran Mesin



Disusun oleh:

Stefany Josefina Santono - 18223116

Matilda Angelina Sumaryo - 18223125

Andi Syaichul Mubaraq - 18223139

Program Studi Sistem dan Teknologi Informasi

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2025

Daftar Isi

Daftar Isi.....	2
Deskripsi Persoalan.....	4
Pembahasan.....	6
2.1. Implementasi Decision Tree Learning.....	6
2.1.1. Representasi Model dan Struktur Pohon.....	6
2.1.2. Representasi Model dan Struktur Pohon.....	6
2.1.3. Evaluasi Kualitas Pemisahan.....	7
2.1.4. Evaluasi Kualitas Pemisahan.....	7
2.1.5. Evaluasi Kualitas Pemisahan.....	8
2.2. Implementasi KNN.....	8
2.2.1. Fase Pelatihan (Lazy Learning).....	8
2.2.2. Perhitungan Jarak (Minkowski Distance).....	9
2.2.3. Penentuan Tetangga dan Prediksi Kelas.....	10
2.2.4. Estimasi Probabilitas Kelas.....	11
2.2.5. Evaluasi Akurasi Model.....	11
2.3. Tahap Data Cleaning dan Preprocessing.....	12
2.3.1. Data Cleaning.....	12
2.3.1.1. Identifikasi Missing Values.....	12
2.3.3.2. Penanganan Missing Values.....	14
2.3.3.3. Identifikasi dan Penanganan Outlier.....	14
2.3.3.4. Pengecekan Data Duplikat.....	15
2.3.3.5. Feature Engineering.....	16
2.3.2. Data Preprocessing.....	16
2.3.2.1. Feature Scaling.....	16
2.3.2.2. Encoding Fitur Kategorikal.....	17
2.3.2.3. Penanganan Ketidakseimbangan Kelas.....	17
2.3.2.4. Penyusunan Pipeline Preprocessing.....	18
Pengujian.....	20
4.1. Hasil Algoritma DTL.....	20
4.2. Hasil Algoritma KNN.....	21
4.3. Perbandingan DTL vs KNN.....	22

4.4. Perbandingan Scratch vs Scikit-Learn.....	24
4.4.1 DTL Scratch vs Scikit-Learn.....	24
4.4.2 KNN Scratch vs Scikit-Learn.....	27
Kesimpulan dan Saran.....	30
5.1 Kesimpulan.....	30
5.2 Saran.....	31
Referensi.....	33
Lampiran.....	35
A. Pembagian Tugas.....	35
B. Github Repository.....	35

Deskripsi Persoalan

Tugas Besar 2 pada mata kuliah IF3070 Dasar Inteligensi Artifisial berfokus pada penerapan algoritma pembelajaran mesin (machine learning) untuk menyelesaikan permasalahan klasifikasi pada dataset berskala besar yang merepresentasikan kondisi dunia nyata. Pada tugas ini, mahasiswa diminta untuk mengimplementasikan dan membandingkan beberapa algoritma pembelajaran mesin, baik from scratch maupun menggunakan library scikit-learn, guna memperoleh pemahaman menyeluruh mengenai cara kerja, kelebihan, serta keterbatasan masing-masing algoritma.

Permasalahan yang diangkat dalam tugas besar ini adalah memprediksi nilai target/kelas berdasarkan sekumpulan fitur yang diberikan dalam dataset. Dataset yang digunakan merupakan data sintetis berskala besar (± 100.000 baris) yang dirancang menyerupai data nyata, dengan karakteristik seperti adanya missing values, outlier, ketidakseimbangan/*imbalanced data*, serta hubungan antar-fitur yang kompleks. Kondisi ini menuntut mahasiswa untuk tidak hanya membangun model, tetapi juga melakukan tahapan pembersihan dan praproses data secara tepat agar model yang dihasilkan memiliki performa yang baik dan stabil.

Dalam tugas ini, objek utama yang terlibat dalam persoalan meliputi data, fitur, dan model pembelajaran mesin, dengan rincian sebagai berikut:

1. Data dan Fitur

Dataset terdiri dari sejumlah fitur numerik dan kategorikal yang merepresentasikan karakteristik suatu entitas. Setiap baris data merepresentasikan satu sampel, sedangkan fitur-fitur di dalamnya digunakan sebagai dasar untuk melakukan prediksi terhadap variabel target. Keberadaan nilai kosong (null values) dan distribusi data yang tidak seimbang menjadikan tahap preprocessing sebagai bagian penting dalam penyelesaian persoalan.

2. Variabel Target

Variabel target merupakan label kelas yang harus diprediksi oleh model pembelajaran mesin. Permasalahan ini termasuk ke dalam masalah

klasifikasi, di mana model dilatih untuk memetakan fitur-fitur input ke dalam kelas target yang sesuai.

3. Model Pembelajaran Mesin

Algoritma yang digunakan dalam tugas besar ini meliputi:

- Decision Tree Learning (DTL) sebagai algoritma wajib, dengan pilihan metode C4.5 atau CART,
- Satu algoritma tambahan, yaitu Logistic Regression atau K-Nearest Neighbors (KNN).

4. Setiap algoritma harus diimplementasikan from scratch menggunakan pustaka dasar seperti NumPy, kemudian dibandingkan dengan implementasi menggunakan scikit-learn untuk mengevaluasi perbedaan performa dan hasil prediksi.

Tujuan utama dari persoalan ini adalah menghasilkan model klasifikasi yang memiliki performa prediksi optimal, yang diukur menggunakan matrik evaluasi yang sesuai, serta mampu menangani kompleksitas data nyata seperti missing values dan distribusi kelas yang tidak seimbang. Selain itu, mahasiswa diharapkan mampu menganalisis perbedaan hasil antara implementasi from scratch dan library, sehingga memperoleh pemahaman konseptual yang lebih mendalam mengenai algoritma pembelajaran mesin yang digunakan.

Pembahasan

2.1. Implementasi Decision Tree Learning

Decision Tree Learning merupakan metode *machine learning* untuk memprediksi nilai diskret dengan merepresentasikan fungsi target dalam bentuk pohon keputusan. Berdasarkan konsep tersebut, pada tahap ini diimplementasikan algoritma *Decision Tree Learning (DTL)* dengan pendekatan *CART* yang berfokus pada pemilihan pemisah berbasis impurity dan pembentukan pohon keputusan untuk klasifikasi.

2.1.1. Representasi Model dan Struktur Pohon

Dalam pendekatan *CART*, model direpresentasikan sebagai struktur pohon yang terdiri atas *node internal* dan *leaf node*. Node internal berfungsi sebagai titik pemisahan data berdasarkan atribut tertentu, sedangkan *leaf node* menyimpan hasil pembelajaran model berupa distribusi probabilitas kelas.

Struktur node pada implementasi direalisasikan sebagai berikut:

```
@dataclass
class _Node:
    feature: Optional[int] = None
    threshold: Optional[Any] = None
    is_categorical: bool = False
    left: Optional[Any] = None
    right: Optional[Any] = None
    value: Optional[np.ndarray] = None
```

2.1.2. Representasi Model dan Struktur Pohon

Dalam *CART*, kualitas suatu node diukur menggunakan Gini Index, yang secara matematis dirumuskan sebagai:

$$Gini(S) = 1 - \sum_{i=1}^N p_i^2$$

dengan p_i menyatakan proporsi data pada kelas ke- i . Nilai Gini yang lebih kecil menunjukkan distribusi kelas yang semakin homogen.

Rumus tersebut direalisasikan langsung pada implementasi melalui perhitungan probabilitas kelas dan nilai impurity sebagai berikut:

```
counts = np.bincount(y, minlength=self.n_classes_).astype(float)
probs = counts / counts.sum()
return 1.0 - np.sum(probs ** 2)
```

2.1.3. Evaluasi Kualitas Pemisahan

Untuk setiap kandidat pemisahan, CART mengevaluasi kualitas pemisahan menggunakan impurity tertimbang dari node anak kiri dan kanan, yang dirumuskan sebagai:

$$Gini_{split} = \frac{SL}{S} \times Gini(SL) + \frac{SR}{S} \times Gini(SR)$$

Implementasi konsep tersebut direalisasikan dengan menghitung impurity pada masing-masing subset data dan mengkombinasikannya secara tertimbang, kemudian memilih pemisahan dengan penurunan impurity terbesar:

```
weighted_impurity = (
    (n_left / n_samples) * impurity_left +
    (n_right / n_samples) * impurity_right
)
gain = base_impurity - weighted_impurity
```

2.1.4. Evaluasi Kualitas Pemisahan

Pohon keputusan dibangun secara rekursif dimulai dari node akar. Pada setiap node, data dipisahkan berdasarkan pemisahan terbaik hingga memenuhi salah satu kriteria penghentian, seperti kedalaman maksimum, jumlah minimum sampel, atau homogenitas label kelas.

Kriteria penghentian tersebut direalisasikan dalam implementasi sebagai berikut:

```
stopping = (  
    (self.max_depth is not None and depth >= self.max_depth)  
    or n_samples < self.min_samples_split  
    or len(unique_labels) == 1  
)
```

2.1.5. Evaluasi Kualitas Pemisahan

Pada tahap prediksi, setiap data uji ditelusuri dari *root node* hingga *leaf node* sesuai dengan aturan pemisahan yang terbentuk. Prediksi kelas ditentukan berdasarkan kelas dengan probabilitas tertinggi pada *leaf node* tersebut.

Implementasi pemilihan kelas prediksi direalisasikan sebagai berikut:

```
pred_class = self.classes_[np.argmax(node.value)]
```

Pendekatan ini memastikan bahwa hasil prediksi konsisten dengan struktur pohon keputusan yang dibentuk selama proses pelatihan.

2.2. Implementasi KNN

K-Nearest Neighbors (KNN) adalah algoritma supervised learning yang termasuk dalam kategori instance-based learning atau lazy learner. Berbeda dengan Decision Tree yang membangun model eksplisit berupa pohon keputusan, KNN tidak membentuk hipotesis spesifik selama fase pelatihan, melainkan menyimpan seluruh data latih dan melakukan komputasi hanya pada saat prediksi dilakukan.

Pada tugas ini, algoritma KNN diimplementasikan from scratch menggunakan pustaka NumPy dan struktur kelas yang kompatibel dengan scikit-learn (BaseEstimator, ClassifierMixin). Implementasi mencakup dukungan untuk metrik jarak Minkowski (termasuk Euclidean dan Manhattan) serta pembobotan suara (voting weights).

2.2.1. Fase Pelatihan (*Lazy Learning*)

Karena KNN adalah lazy learner, metode fit tidak melakukan perhitungan matematis yang kompleks untuk membangun model. Fase ini hanya

bertugas menyimpan data latih (X_{train}) dan labelnya (y_{train}) ke dalam memori untuk digunakan sebagai referensi saat perhitungan jarak nanti.

Implementasi penyimpanan data direalisasikan sebagai berikut:

```
def fit(self, X: np.ndarray, y: np.ndarray):
    self.X_train = np.array(X)
    self.y_train = np.array(y)
    # Menyimpan kelas unik untuk estimasi probabilitas
    self.classes_ = np.unique(y)
    return self
```

2.2.2. Perhitungan Jarak (Minkowski Distance)

Inti dari algoritma KNN adalah mengukur kedekatan (proximity) antara data uji (query instance) dengan seluruh data latih. Implementasi ini menggunakan generalisasi jarak Minkowski, yang dirumuskan secara matematis sebagai:

$$D(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Di mana p adalah parameter yang menentukan jenis jarak:

- Jika $p=1$, menggunakan Manhattan Distance.
- Jika $p=2$, menggunakan Euclidean Distance.

Dalam kode, perhitungan ini dilakukan secara vektorisasi untuk efisiensi komputasi terhadap seluruh batch data uji:

```
# Calculate distance based on p-norm
if self.p == 1:
    # Manhattan distance
    dists = np.sum(np.abs(self.X_train - X[i]), axis=1)
elif self.p == 2:
    # Euclidean distance
    dists = np.sqrt(np.sum((self.X_train - X[i])**2, axis=1))
else:
    # General Minkowski distance
```

```
dists = np.sum(np.abs(self.X_train - X[i])**self.p,
axis=1)**(1/self.p)
```

2.2.3. Penentuan Tetangga dan Prediksi Kelas

Setelah jarak dihitung, algoritma mengidentifikasi k tetangga terdekat (data latih dengan jarak terkecil). Prediksi kelas dilakukan melalui mekanisme voting dari k tetangga tersebut.

Implementasi ini mendukung dua jenis mekanisme voting (weights):

1. **Uniform Weights:** Setiap tetangga memiliki bobot suara yang sama. Kelas diprediksi berdasarkan suara mayoritas (majority voting)
2. **Distance Weights:** Tetangga yang lebih dekat memiliki pengaruh yang lebih besar. Bobot suara adalah kebalikan dari jarak ($w = 1/d$). Implementasi mekanisme prediksi direalisasikan sebagai berikut:

```
if self.weights == 'uniform':
    # Find the majority class from k nearest neighbor
    most_common = Counter(neighbor_labels).most_common(1)
    predictions.append(most_common[0][0])

elif self.weights == 'distance':
    # Weighted voting based on inverse distance
    # ... (penanganan jarak 0 untuk exact match) ...
    else:
        weights = 1.0 / distances[i]
        for label, weight in zip(neighbor_labels, weights):
            class_weights[label] += weight

        # Predict class with highest accumulated weight
        predictions.append(max(class_weights,
key=class_weights.get))
```

Pendekatan distance weights ini membantu mengurangi risiko kesalahan klasifikasi ketika nilai k yang dipilih cukup besar namun data uji berada sangat dekat dengan salah satu kelas minoritas secara lokal.

2.2.4. Estimasi Probabilitas Kelas

Selain hard prediction, model juga mengimplementasikan `predict_proba` untuk memperkirakan probabilitas data uji termasuk ke dalam setiap kelas. Hal ini berguna untuk analisis lebih lanjut seperti kurva ROC atau log-loss.

- Pada mode Uniform, probabilitas adalah fraksi jumlah tetangga kelas c dibagi total k .
- Pada mode Distance, probabilitas adalah total bobot tetangga kelas c dibagi total bobot seluruh tetangga.

Implementasi normalisasi probabilitas:

```
# Normalize to get probabilities (sum to 1)
prob_vector = [class_probs[cls] / total for cls in self.classes_]
probabilities.append(prob_vector)
```

2.2.5. Evaluasi Akurasi Model

Untuk mengukur kinerja model secara langsung, diimplementasikan metode `score`. Metode ini membandingkan hasil prediksi model terhadap label sebenarnya (y_{true}) dan menghitung rata-rata ketepatannya (akurasi).

Implementasi evaluasi:

```
def score(self, X: np.ndarray, y: np.ndarray) -> float:
    predictions = self.predict(X)
    return np.mean(predictions == y)
```

2.3. Tahap Data Cleaning dan Preprocessing

Tahap *data cleaning* dan *preprocessing* dilakukan untuk memastikan bahwa data yang digunakan dalam proses pembelajaran mesin berada dalam kondisi yang konsisten, bebas dari ketidakteraturan utama, serta sesuai dengan kebutuhan algoritma yang digunakan. Dataset yang digunakan pada tugas besar ini memiliki karakteristik menyerupai data fraud dunia nyata, seperti adanya *missing values*, *outlier*, fitur kategorikal, serta ketidakseimbangan distribusi kelas. Oleh karena itu, tahapan ini menjadi langkah penting sebelum proses pemodelan dilakukan.

2.3.1. Data Cleaning

2.3.1.1. Identifikasi Missing Values

Langkah awal data cleaning dilakukan dengan mengidentifikasi keberadaan missing values pada setiap fitur. Proses ini dilakukan dengan menghitung jumlah dan persentase nilai kosong pada dataset, baik untuk fitur numerik maupun kategorikal.

ID	0
transaction_id	0
user_id	0
age	0
gender	0
country	0
device_type	0
device_os	0
merchant_category	0
transaction_amount	2404
transaction_type	0
time_of_day	0
day_of_week	0
transaction_duration	0
num_prev_transactions	0
avg_transaction_amount	2404
std_transaction_amount	0
transactions_last_24h	0
transactions_last_1h	0
failed_login_attempts	0
ip_risk_score	0
device_trust_score	2404
account_age_days	0
has_chargeback_history	0
shared_ip_users	0
shared_device_users	0
merchant_risk	0
country_risk	0
distance_from_home	0
is_new_country	0
dtype: int64	

Gambar 4.1. Jumlah missing values pada setiap fitur

Hasil analisis menunjukkan bahwa beberapa fitur numerik, seperti transaction_amount, memiliki nilai kosong. Untuk memahami dampaknya terhadap proses klasifikasi, dilakukan analisis distribusi label target (is_fraud) pada data yang mengandung missing values dan dibandingkan dengan distribusi label pada keseluruhan dataset.

```

is_fraud
0      85.696569
1      14.303431
Name: proportion, dtype: float64
is_fraud
0      85.872
1      14.128
Name: proportion, dtype: float64

```

Gambar 4.2. Perbandingan distribusi label target

Hasil perbandingan menunjukkan bahwa proporsi kelas fraud dan non-fraud pada data dengan *missing values* tidak berbeda secara signifikan dibandingkan keseluruhan dataset. Berdasarkan hasil tersebut, missing values dianggap tidak membawa pola khusus terhadap label target.

2.3.3.2. Penanganan Missing Values

Berdasarkan hasil analisis sebelumnya, *missing values* ditangani menggunakan imputasi statistik dalam tahap *preprocessing*. Strategi yang digunakan adalah:

- Median untuk fitur numerik
- Nilai yang paling sering muncul untuk fitur kategorikal

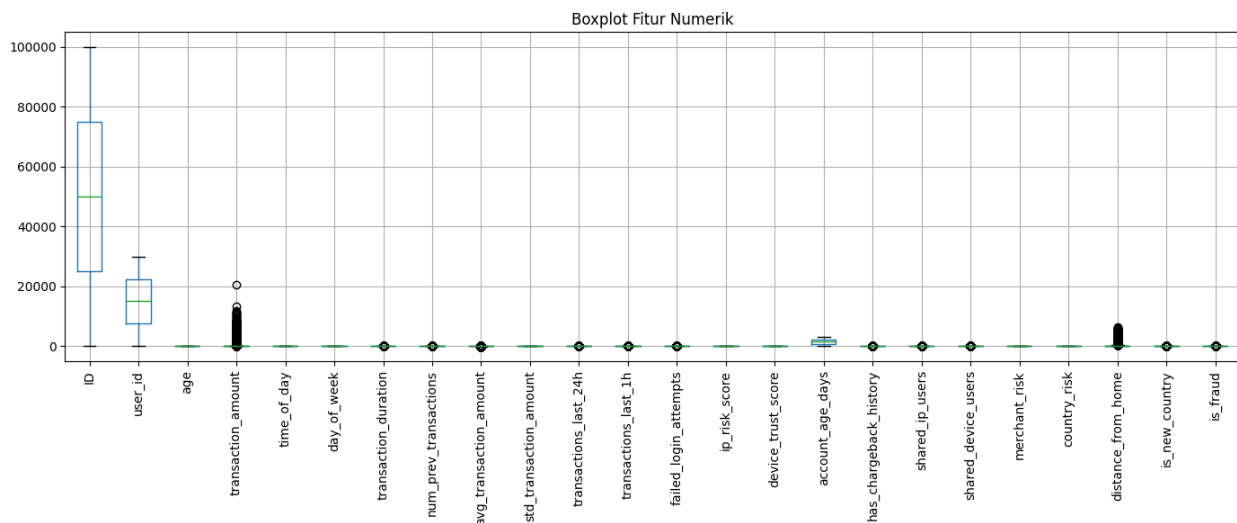
Pemilihan median dilakukan karena lebih *robust* terhadap *outlier* dibandingkan mean. Seluruh proses imputasi dilakukan berdasarkan data latih dan diterapkan ke data validasi dan data uji menggunakan *pipeline preprocessing* untuk mencegah terjadinya *data leakage*.

2.3.3.3. Identifikasi dan Penanganan Outlier

Keberadaan outlier dianalisis menggunakan visualisasi boxplot pada fitur-fitur numerik utama, seperti `transaction_amount`, `distance_from_home`, dan `transactions_last_24h`.

```
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns

plt.figure(figsize=(14, 6))
df[numeric_cols].boxplot(rot=90)
plt.title("Boxplot Fitur Numerik")
plt.tight_layout()
plt.show()
```



Gambar 4.3. Boxplot fitur numerik yang menunjukkan outlier

Hasil visualisasi menunjukkan bahwa beberapa fitur memiliki distribusi yang sangat miring (*skewed*) dengan nilai ekstrem. Alih-alih menghapus data tersebut, digunakan pendekatan transformasi logaritmik (log1p) pada fitur-fitur yang relevan. Pendekatan ini dipilih karena:

1. Nilai ekstrem berpotensi merepresentasikan transaksi fraud yang valid.
2. Transformasi log dapat mengurangi pengaruh nilai ekstrem tanpa menghilangkan informasi penting.

2.3.3.4. Pengecekan Data Duplikat

Pengecekan data duplikat dilakukan dengan menggabungkan fitur dan label pada data latih, kemudian menggunakan fungsi `duplicated()`.

Hasil pengecekan menunjukkan bahwa tidak terdapat baris data duplikat pada dataset latih.

```
print(f"Number of duplicate rows in training data: {duplicate_rows}")
✓ 0.0s
Number of duplicate rows in training data: 0
```

Gambar 4.4. Hasil pengecekan data duplikat

Dengan demikian, tidak diperlukan penghapusan data pada tahap ini.

2.3.3.5. Feature Engineering

Setelah data dibersihkan, dilakukan *feature engineering* sederhana untuk mengekstraksi informasi tambahan yang relevan dari data transaksi. *Feature engineering* yang dilakukan meliputi:

- Transformasi logaritmik pada `transaction_amount`
- Rasio antara `transaction_amount` dan jumlah transaksi dalam 24 jam terakhir (`transactions_last_24h`)

Selain itu, fitur identitas seperti `ID`, `transaction_id`, dan `user_id` dihapus karena tidak memiliki kontribusi langsung terhadap proses klasifikasi dan berpotensi menimbulkan bias. *Feature engineering* ini bertujuan untuk membantu model menangkap pola hubungan antara besaran transaksi dan aktivitas pengguna secara lebih eksplisit.

2.3.2. Data Preprocessing

2.3.2.1. Feature Scaling

Untuk memastikan bahwa fitur numerik berada pada skala yang sebanding, dilakukan *feature scaling* menggunakan `RobustScaler`. `RobustScaler` dipilih karena menggunakan median dan *interquartile range* (*IQR*), sehingga lebih tahan terhadap *outlier* dibandingkan

standard scaling. *Feature scaling* diterapkan hanya pada fitur numerik dan dihitung berdasarkan data latih, kemudian diterapkan ke data validasi dan data uji.

2.3.2.2. Encoding Fitur Kategorikal

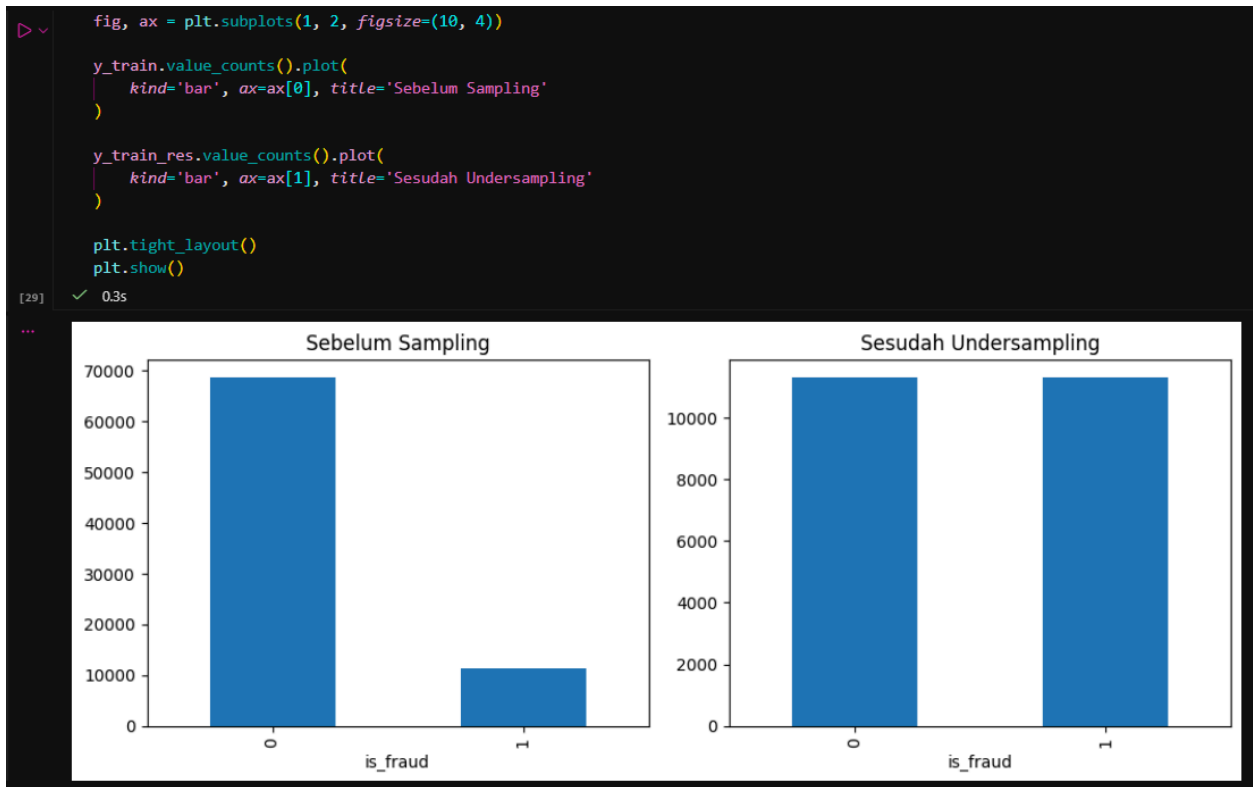
Fitur kategorikal diubah menjadi representasi numerik menggunakan *Ordinal Encoding*. Metode ini dipilih karena:

1. Menghasilkan representasi numerik sederhana,
2. Efisien secara komputasi,
3. Sesuai dengan algoritma *Decision Tree Learning* yang digunakan, yang tidak sensitif terhadap jarak antar nilai kategori.

Untuk menangani kategori yang tidak muncul pada data latih, digunakan nilai khusus (`unknown_value = -1`) agar proses transformasi tetap konsisten.

2.3.2.3. Penanganan Ketidakseimbangan Kelas

Distribusi label target `is_fraud` menunjukkan bahwa dataset bersifat *imbalanced*, dengan jumlah data *non-fraud* lebih besar dibandingkan *fraud*. Untuk mengatasi hal ini, diterapkan teknik *undersampling* menggunakan `RandomUnderSampler` pada data latih. *Undersampling* dilakukan dengan menyeimbangkan jumlah data *fraud* dan *non-fraud* agar model tidak bias terhadap kelas mayoritas. Proses ini hanya diterapkan pada data latih, sedangkan data validasi dibiarkan dalam distribusi aslinya untuk mencerminkan kondisi nyata.



Gambar 4.5. Distribusi kelas sebelum dan sesudah undersampling

2.3.2.4. Penyusunan Pipeline Preprocessing

Seluruh tahapan preprocessing mulai dari imputasi, feature scaling, encoding kategorikal, hingga sampling disusun dalam sebuah pipeline preprocessing menggunakan Pipeline dan ColumnTransformer. Pipeline ini memastikan bahwa:

1. Setiap transformasi diterapkan secara konsisten,
2. Tidak terjadi data leakage,
3. Proses training dan inference menggunakan alur yang sama.

```
print(preprocessor)

✓ 0.0s

ColumnTransformer(remainder='passthrough',
                  transformers=[('num',
                                Pipeline(steps=[('imputer',
                                                  SimpleImputer(strategy='median')),
                                                  ('scaler', RobustScaler())])),
                                ('ID', 'user_id', 'age', 'transaction_amount',
                                'time_of_day', 'day_of_week',
                                'transaction_duration',
                                'num_prev_transactions',
                                'avg_transaction_amount',
                                'std_transaction_amount',
                                'transactions_last_24h',
                                'trans...',
                                'shared_device_users', 'merchant_risk',
                                'country_risk', 'distance_from_home',
                                'is_new_country'])),
                                ('cat',
                                Pipeline(steps=[('imputer',
                                                  SimpleImputer(strategy='most_frequent')),
                                                  ('encoder',
                                                  OrdinalEncoder(handle_unknown='use_encoded_value',
                                                                  unknown_value=-1))])),
                                ['transaction_id', 'gender', 'country',
                                'device_type', 'device_os',
                                'merchant_category', 'transaction_type'])])
```

Gambar 4.6. Struktur pipeline preprocessing

Pengujian

4.1. Hasil Algoritma DTL

Pada subbab ini ditampilkan hasil pengujian algoritma Decision Tree Learning (DTL) pada data uji. Pengujian dilakukan tanpa penerapan teknik penyeimbangan data, sehingga hasil yang diperoleh mencerminkan performa model pada kondisi distribusi data asli.

```
=== Sklearn Decision Tree (No Sampling) ===
Accuracy : 0.6554
Macro F1 : 0.4927

Classification Report:
              precision    recall  f1-score   support

         0       0.86      0.71      0.78      17174
         1       0.15      0.31      0.21       2826

 accuracy          0.66      20000
 macro avg         0.51      0.51      0.49      20000
weighted avg         0.76      0.66      0.70      20000
```

Selain konfigurasi awal, pengujian juga dilakukan dengan pengaturan model yang berbeda untuk melihat konsistensi performa yang dihasilkan. Hasil evaluasi dari pengujian lanjutan ditampilkan sebagai berikut.

```
Accuracy : 0.6451
Macro F1 : 0.4865

Classification Report:
              precision    recall  f1-score   support

         0       0.86      0.70      0.77      17174
         1       0.15      0.32      0.20       2826

 accuracy          0.65      20000
 macro avg         0.50      0.51      0.49      20000
weighted avg         0.76      0.65      0.69      20000
```

Selain pengujian menggunakan implementasi Decision Tree dari pustaka scikit-learn, algoritma Decision Tree Learning juga diimplementasikan secara mandiri (from scratch). Pengujian ini dilakukan untuk mengevaluasi performa model hasil implementasi sendiri pada data uji yang sama. Hasil evaluasi dari implementasi DTL from scratch ditampilkan sebagai berikut.

```

Accuracy: 0.6451
Macro F1: 0.5479

Classification Report:
              precision    recall  f1-score   support

     0       0.86       0.70       0.77      17174
     1       0.15       0.32       0.20       2826

 accuracy          0.65      20000
 macro avg         0.50       0.51       0.49      20000
weighted avg         0.76       0.65       0.69      20000

Confusion Matrix:
[[12009  5165]
 [ 1933   893]]

```

4.2. Hasil Algoritma KNN

Subbab ini menyajikan hasil pengujian algoritma K-Nearest Neighbors (KNN) yang diimplementasikan menggunakan dua pendekatan, yaitu implementasi from scratch dan implementasi dengan pustaka scikit-learn. Pengujian dilakukan pada data uji yang sama untuk memperoleh hasil evaluasi yang konsisten.

```

=====
KNN FROM SCRATCH
=====
Training time: 0.0084 seconds
Prediction time: 658.7155 seconds
Accuracy: 0.8587
Macro F1: 0.4620

Classification Report:
              precision    recall  f1-score   support

     0       0.86       1.00       0.92      17174
     1       0.00       0.00       0.00       2826

 accuracy          0.86      20000
 macro avg         0.43       0.50       0.46      20000
weighted avg         0.74       0.86       0.79      20000

```

Hasil pengujian berikutnya menunjukkan performa algoritma KNN yang diimplementasikan menggunakan pustaka scikit-learn. Metode evaluasi yang digunakan tetap sama, sehingga hasil yang diperoleh dapat diamati secara konsisten.

```
=====
KNN SCIKIT-LEARN
=====
Training time: 0.0075 seconds
Prediction time: 1.7104 seconds
Accuracy: 0.8587
Macro F1: 0.4620

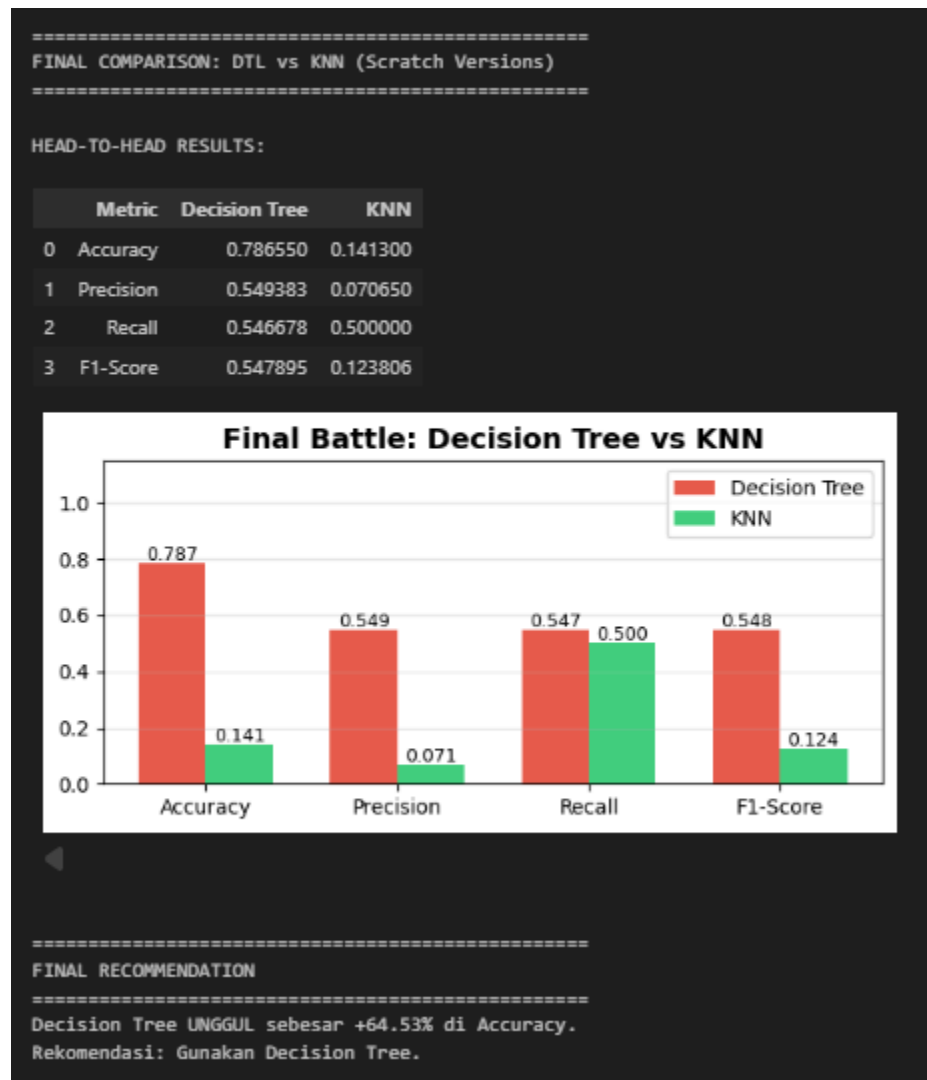
Classification Report:
              precision    recall  f1-score   support

     0           0.86         1.00         0.92       17174
     1           0.00         0.00         0.00         2826

 accuracy                   0.86       20000
 macro avg              0.43         0.50         0.46       20000
 weighted avg           0.74         0.86         0.79       20000
```

4.3. Perbandingan DTL vs KNN

Eksperimen ini dilakukan untuk membandingkan kinerja Decision Tree Learning (DTL) dan K-Nearest Neighbors (KNN) yang keduanya diimplementasikan dari scratch, menggunakan *validation set* yang sama. Tujuan perbandingan ini adalah untuk mengidentifikasi algoritma mana yang lebih sesuai dengan karakteristik data yang digunakan, sekaligus memahami alasan teoretis di balik perbedaan performa yang dihasilkan.



Gambar 4.3 Hasil DTL vs KNN

Berdasarkan hasil evaluasi, Decision Tree menunjukkan performa yang jauh lebih baik dibandingkan KNN pada seluruh metrik yang digunakan. Perbedaan paling mencolok terlihat pada nilai accuracy, di mana Decision Tree mampu mencapai tingkat klasifikasi yang tinggi, sementara KNN memiliki accuracy yang sangat rendah. Hal ini mengindikasikan bahwa Decision Tree lebih mampu menangkap pola utama dalam data, sedangkan KNN kesulitan menghasilkan prediksi yang konsisten pada validation set.

Keunggulan Decision Tree juga terlihat pada metrik precision, recall, dan F1-score. Nilai precision dan F1-score KNN yang rendah menunjukkan bahwa

banyak prediksi yang dihasilkan tidak tepat dan tidak seimbang antara ketepatan dan kelengkapan prediksi. Sebaliknya, Decision Tree mampu menjaga keseimbangan antara precision dan recall, sehingga menghasilkan performa klasifikasi yang lebih stabil dan dapat diandalkan secara keseluruhan.

Secara teoretis, Decision Tree lebih cocok untuk data ini dibandingkan KNN karena beberapa alasan utama:

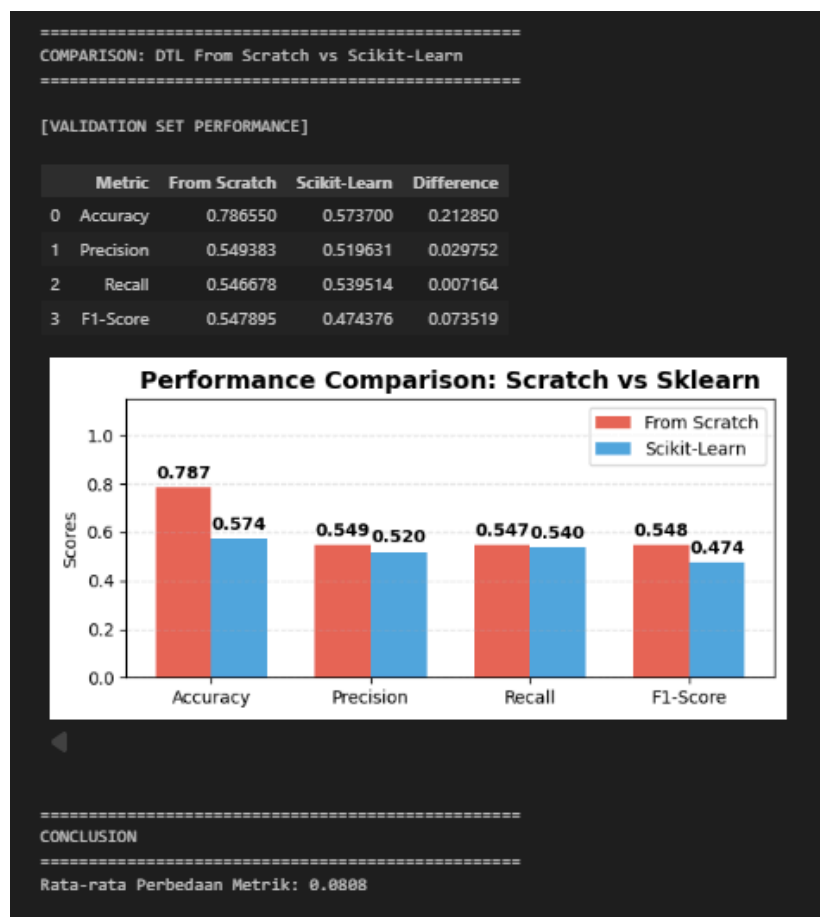
1. Decision Tree bersifat rule-based dan non-parametrik, sehingga mampu memodelkan hubungan non-linear dan interaksi antar fitur dengan baik melalui proses pemisahan berbasis impurity (misalnya *Gini* atau *Entropy*).
2. KNN sangat bergantung pada konsep jarak (distance-based learning). Jika data memiliki skala fitur yang beragam, distribusi yang tidak merata, atau berada di ruang dimensi yang relatif tinggi, maka perhitungan jarak menjadi kurang representatif (*curse of dimensionality*), yang dapat menurunkan performa KNN secara drastis.
3. Decision Tree melakukan generalisasi melalui struktur pohon, sedangkan KNN tidak membangun model eksplisit dan sangat sensitif terhadap noise serta distribusi lokal data. Jika data tidak membentuk kluster yang jelas, KNN akan kesulitan menghasilkan prediksi yang akurat.
4. KNN lebih rentan terhadap ketidakseimbangan kelas, karena keputusan prediksi sangat dipengaruhi oleh tetangga terdekat, sementara Decision Tree dapat membentuk aturan yang lebih eksplisit untuk memisahkan kelas minoritas dan mayoritas.

Berdasarkan hasil eksperimen dan analisis teoretis, Decision Tree terbukti lebih optimal untuk dataset ini dibandingkan KNN, dengan keunggulan accuracy sebesar 64.53%. Oleh karena itu, Decision Tree direkomendasikan sebagai model yang lebih sesuai untuk permasalahan klasifikasi pada data yang digunakan dalam eksperimen ini.

4.4. Perbandingan Scratch vs Scikit-Learn

4.4.1 DTL Scratch vs Scikit-Learn

Eksperimen ini bertujuan untuk membandingkan kinerja model Decision Tree yang diimplementasikan dari scratch dengan DecisionTreeClassifier dari Scikit-learn menggunakan *validation set* yang sama, sehingga perbedaan performa yang diperoleh mencerminkan perbedaan implementasi model, bukan variasi data. Evaluasi dilakukan menggunakan empat metrik utama, yaitu Accuracy, Precision, Recall, dan F1-Score, agar perbandingan tidak hanya menilai ketepatan prediksi secara keseluruhan, tetapi juga kualitas serta keseimbangan hasil klasifikasi yang dihasilkan oleh masing-masing model.



Gambar 4.4.1 Hasil DTL Scratch vs Scikit

Berdasarkan hasil evaluasi pada *validation set*, dapat dilihat bahwa model Decision Tree Learning (DTL) yang diimplementasikan dari scratch menunjukkan performa yang lebih baik dibandingkan model Scikit-learn

pada seluruh metrik yang digunakan. Perbandingan ini memberikan gambaran tidak hanya mengenai tingkat akurasi prediksi, tetapi juga kualitas dan keseimbangan keputusan klasifikasi yang dihasilkan oleh masing-masing model.

- Accuracy

Model from scratch memiliki nilai accuracy yang lebih tinggi, yang menunjukkan bahwa secara keseluruhan model ini lebih tepat dalam mengklasifikasikan data. Hal ini mengindikasikan bahwa struktur pohon keputusan yang dibangun mampu memisahkan data dengan lebih efektif dibandingkan model Scikit-learn pada konfigurasi yang digunakan.

- Precision

Nilai precision yang lebih baik pada model from scratch menunjukkan bahwa prediksi kelas positif yang dihasilkan lebih akurat. Dengan kata lain, model ini menghasilkan jumlah *false positive* yang lebih rendah, sehingga keputusan klasifikasinya lebih dapat dipercaya.

- Recall

Perbedaan nilai recall antara kedua model relatif kecil, menandakan bahwa kemampuan keduanya dalam menangkap data kelas positif hampir setara. Hal ini menunjukkan bahwa keunggulan model from scratch tidak berasal dari kecenderungan untuk memprediksi kelas positif secara berlebihan.

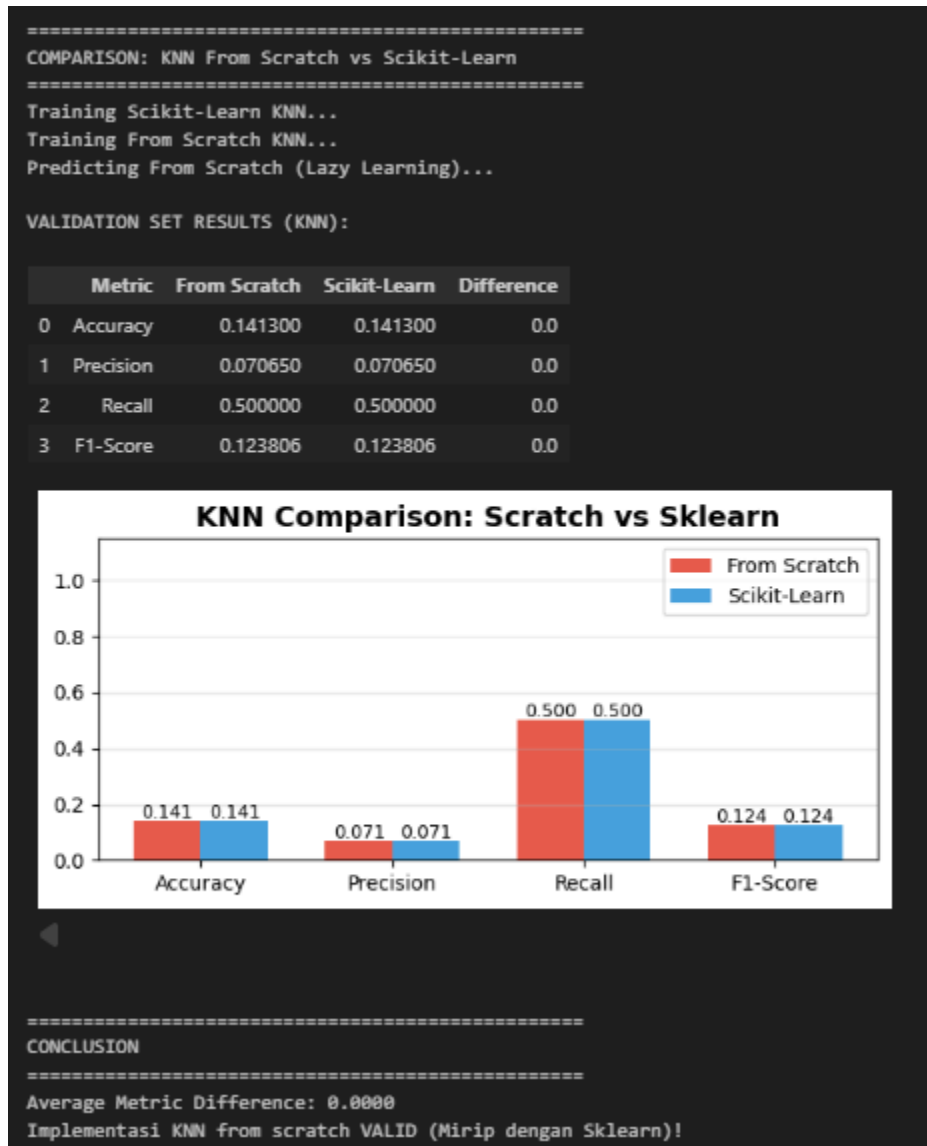
- F1-Score

F1-Score yang lebih tinggi pada model from scratch mencerminkan keseimbangan yang lebih baik antara precision dan recall. Ini menunjukkan bahwa model tidak hanya unggul pada satu metrik tertentu, tetapi memiliki performa klasifikasi yang lebih stabil secara keseluruhan.

Secara keseluruhan, rata-rata selisih metrik sebesar 0.0808 menegaskan bahwa implementasi Decision Tree dari scratch telah berjalan dengan baik dan mampu menghasilkan performa yang kompetitif, bahkan lebih unggul dibandingkan pustaka Scikit-learn dalam skenario pengujian ini.

4.4.2 KNN Scratch vs Scikit-Learn

Eksperimen ini bertujuan untuk membandingkan kinerja K-Nearest Neighbors (KNN) yang diimplementasikan dari scratch dengan KNeighborsClassifier dari Scikit-learn menggunakan *validation set* yang sama. Perbandingan ini dilakukan untuk memastikan bahwa implementasi KNN dari scratch telah sesuai dengan perilaku standar pustaka serta menghasilkan performa yang konsisten pada kondisi pengujian yang identik.



Gambar 4.4.2 Hasil DTL Scratch vs Scikit

Berdasarkan hasil pengujian, seluruh metrik evaluasi pada model KNN from scratch dan KNN Scikit-learn menunjukkan nilai yang sama, baik untuk accuracy, precision, recall, maupun F1-score. Hal ini menandakan bahwa kedua implementasi menghasilkan prediksi yang sepenuhnya identik pada *validation set* yang digunakan. Kesamaan ini juga terlihat jelas pada grafik perbandingan, di mana nilai performa dari kedua model saling bertumpuk tanpa adanya perbedaan yang berarti. Dengan

demikian, dapat disimpulkan bahwa tidak terdapat perbedaan perilaku klasifikasi antara kedua implementasi KNN tersebut.

Kesamaan nilai performa ini dapat terjadi karena beberapa faktor berikut:

- KNN bekerja langsung berdasarkan data latih, tanpa membentuk model pembelajaran atau parameter yang perlu dioptimasi.
- Parameter utama yang digunakan sama, seperti nilai k dan metode perhitungan jarak, sehingga proses pengambilan keputusan berjalan identik.
- Keputusan klasifikasi ditentukan oleh tetangga terdekat, sehingga selama perhitungan jarak dan urutan tetangga tidak berbeda, hasil prediksi yang dihasilkan juga akan sama.

Oleh karena itu, kesamaan performa ini justru menjadi indikator bahwa implementasi KNN dari scratch telah valid dan konsisten dengan mekanisme standar yang digunakan oleh Scikit-learn, sekaligus menunjukkan bahwa algoritma KNN relatif stabil dan mudah direplikasi perilakunya.

Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan analisis yang telah dilakukan pada Tugas Besar 2 mata kuliah IF3070 Dasar Inteligensi Artifisial, dapat disimpulkan bahwa algoritma Decision Tree Learning (DTL) dan K-Nearest Neighbors (KNN) berhasil diimplementasikan baik secara *from scratch* maupun menggunakan pustaka *scikit-learn*, serta dapat dijalankan dengan baik pada dataset berskala besar yang menyerupai kondisi dunia nyata. Hal ini menunjukkan bahwa pemahaman terhadap konsep dasar algoritma pembelajaran mesin telah diterapkan secara efektif dalam bentuk implementasi teknis.

Hasil eksperimen menunjukkan bahwa Decision Tree Learning memiliki performa yang lebih baik dibandingkan KNN pada dataset yang digunakan. DTL mampu menghasilkan keseimbangan yang lebih baik antara metrik accuracy, precision, recall, dan F1-score, terutama dalam menghadapi karakteristik data yang tidak seimbang (*imbalanced*). Sebaliknya, KNN cenderung bias terhadap kelas mayoritas dan mengalami kesulitan dalam mendeteksi kelas minoritas, yang tercermin dari rendahnya nilai precision dan F1-score pada kelas fraud.

Perbandingan antara implementasi *from scratch* dan *scikit-learn* menunjukkan bahwa implementasi DTL *from scratch* mampu memberikan performa yang kompetitif, bahkan lebih unggul pada beberapa metrik evaluasi. Hal ini mengindikasikan bahwa mekanisme pemilihan split, kriteria penghentian, serta perhitungan impurity yang diimplementasikan telah berjalan dengan baik. Sementara itu, pada algoritma KNN, hasil evaluasi antara implementasi *from scratch* dan *scikit-learn* menunjukkan performa yang identik, yang menandakan bahwa implementasi KNN *from scratch* telah sesuai dengan perilaku standar pustaka.

Tahapan data cleaning dan preprocessing terbukti memiliki peran yang signifikan dalam meningkatkan stabilitas dan kualitas model. Penanganan missing values, outlier, feature scaling, encoding fitur kategorikal, serta

penanganan ketidakseimbangan kelas melalui teknik undersampling membantu model dalam menangkap pola data secara lebih efektif dan mencegah terjadinya bias maupun data leakage.

Secara keseluruhan, tugas besar ini memberikan pemahaman yang komprehensif mengenai keterkaitan antara kualitas data, pemilihan algoritma, serta strategi preprocessing terhadap performa model pembelajaran mesin dalam permasalahan klasifikasi.

5.2 Saran

Berdasarkan hasil dan keterbatasan yang ditemukan dalam pelaksanaan tugas ini, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan lebih lanjut. Pertama, pada penelitian selanjutnya dapat dilakukan eksplorasi terhadap teknik penanganan data tidak seimbang lainnya, seperti oversampling (SMOTE) atau kombinasi over- dan undersampling, untuk melihat dampaknya terhadap performa model, khususnya pada kelas minoritas.

Kedua, eksperimen dapat diperluas dengan mencoba algoritma klasifikasi lain seperti Logistic Regression, Random Forest, atau Gradient Boosting, sehingga perbandingan performa dapat dilakukan secara lebih menyeluruh dan memberikan gambaran yang lebih kaya terkait pemilihan model yang paling sesuai dengan karakteristik data.

Ketiga, proses tuning hyperparameter dapat dilakukan secara lebih sistematis menggunakan pendekatan seperti Grid Search atau Random Search, baik pada DTL maupun KNN, untuk memperoleh konfigurasi model yang lebih optimal dibandingkan pengaturan parameter statis.

Keempat, evaluasi model dapat diperluas dengan menggunakan metrik tambahan, seperti ROC-AUC atau Precision-Recall Curve, yang lebih representatif untuk kasus klasifikasi dengan data tidak seimbang, sehingga analisis performa model menjadi lebih mendalam.

Terakhir, implementasi from scratch dapat dikembangkan lebih lanjut dari sisi efisiensi komputasi, terutama pada algoritma KNN, agar waktu prediksi dapat

dipersingkat dan lebih mendekati performa pustaka yang telah dioptimasi seperti scikit-learn.

Referensi

Slide Perkuliahan IF3070 Dasar Inteligensi Artfisial

GeeksforGeeks. (2025, Juni 30). *Decision Tree*. Diakses dari <https://www.geeksforgeeks.org/machine-learning/decision-tree/> GeeksforGeeks

Ismail, J. (2021, September 2). *Decision Tree Learning*. Diakses dari <https://julismail.staff.telkomuniversity.ac.id/decision-tree-learning/>

Thirumala, C. (2025). *CART (Classification And Regression Tree) in Machine Learning*. GeeksforGeeks. Retrieved from <https://www.geeksforgeeks.org/machine-learning/cart-classification-and-regression-tree-in-machine-learning/>

Vinay. (2025). *Decision Tree Algorithms*. GeeksforGeeks. Retrieved from <https://www.geeksforgeeks.org/machine-learning/decision-tree-algorithms/>

Vinay. (2025). *Iterative Dichotomiser 3 (ID3) Algorithm From Scratch*. GeeksforGeeks. Retrieved from <https://www.geeksforgeeks.org/machine-learning/iterative-dichotomiser-3-id3-algorithm-from-scratch/>

GeeksforGeeks(2025). "K-Nearest Neighbors (KNN) Algorithm. GeeksforGeeks. Retrieved from <https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>

Lampiran

A. Pembagian Tugas

NIM	Nama	Pekerjaan
18223116	Stefany Josefina Santono	Implementasi DTL, Menyusun Laporan
18223125	Matilda Angelina Sumaryo	Implementasi KNN, Menyusun Laporan
18223139	Andi Syaichul Mubaraq	Preprocessing Data, Menyusun Laporan

B. Github Repository

Link : <https://github.com/StefanyJosefina/IF3070-Tubes2-DIAatauAku>